

Practical Week 9

Aim

To implement a file-copy program using Linux system calls and compare it with a standard C library file-copy program.

Program:



```
GNU nano 8.7.1 copy_syscall.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int source, destination;
    char buffer[1024];
    ssize_t bytesRead, bytesWritten;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = open(argv[1], O_RDONLY);

    if (source < 0)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (destination < 0)
    {
        perror("Error opening destination file");
        close(source);
        return 1;
    }

    while ((bytesRead = read(source, buffer, sizeof(buffer))) > 0)
    {
        bytesWritten = write(destination, buffer, bytesRead);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
        }
    }
}
```

^G Help ^O Write Out ^F Where Is ^K Cut ^T Execute ^C Location
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^_ Go To Line

```

        perror("Error writing file");
        close(source);
        close(destination);
        return 1;
    }
}

if (bytesRead < 0)
{
    perror("Error reading file");
}

close(source);
close(destination);

printf("File copied successfully using system calls.\n");

return 0;
}

```

Output:

```

navaneeth@Navaneeth: ~/pr
navaneeth@Navaneeth:~$ mkdir -p ~/practical_week9
navaneeth@Navaneeth:~$ cd ~/practical_week9
navaneeth@Navaneeth:~/practical_week9$ nano copy_syscall.c
navaneeth@Navaneeth:~/practical_week9$ gcc copy_syscall.c -o copy_syscall
navaneeth@Navaneeth:~/practical_week9$ echo "This is Practical Week 9." > source.txt
navaneeth@Navaneeth:~/practical_week9$ for i in {1..1000}; do echo "Linux file copy using system calls"
" >> source.txt; done
navaneeth@Navaneeth:~/practical_week9$ head source.txt
This is Practical Week 9.
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
Linux file copy using system calls
navaneeth@Navaneeth:~/practical_week9$ ./copy_syscall source.txt copied_syscall.txt
File copied successfully using system calls.
navaneeth@Navaneeth:~/practical_week9$ ls -l source.txt copied_syscall.txt
-rw-r--r-- 1 navaneeth navaneeth 35026 Sep 27 09:15 copied_syscall.txt
-rw-r--r-- 1 navaneeth navaneeth 35026 Sep 27 09:14 source.txt
navaneeth@Navaneeth:~/practical_week9$ cmp source.txt copied_syscall.txt
navaneeth@Navaneeth:~/practical_week9$ ^C
navaneeth@Navaneeth:~/practical_week9$ diff source.txt copied_syscall.txt

```

```
navaneeth@Navaneeth: ~/pr: × + ▾
GNU nano 8.7.1 copy_standard.c *
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    FILE *source, *destination;
    char buffer[1024];
    size_t bytesRead, bytesWritten;

    if (argc != 3)
    {
        printf("Usage: %s <source> <destination>\n", argv[0]);
        return 1;
    }

    source = fopen(argv[1], "rb");

    if (source == NULL)
    {
        perror("Error opening source file");
        return 1;
    }

    destination = fopen(argv[2], "wb");

    if (destination == NULL)
    {
        perror("Error opening destination file");
        fclose(source);
        return 1;
    }

    while ((bytesRead = fread(buffer, 1, sizeof(buffer), source)) > 0)
    {
        bytesWritten = fwrite(buffer, 1, bytesRead, destination);

        if (bytesWritten != bytesRead)
        {
            perror("Error writing file");
            fclose(source);
            fclose(destination);
            return 1;
        }
    }

    fclose(source);
    fclose(destination);

    printf("File copied successfully using standard library functions.\n");

    return 0;
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^_ Justify   ^/ Go To Line M-E Redo
```

```

navaneeth@Navaneeth:~/practical_week9$ nano copy_standard.c
navaneeth@Navaneeth:~/practical_week9$ gcc copy_standard.c -o copy_standard
navaneeth@Navaneeth:~/practical_week9$ ./copy_standard source.txt copied_standard.txt
File copied successfully using standard library functions.
navaneeth@Navaneeth:~/practical_week9$ cmp source.txt copied_standard.txt
navaneeth@Navaneeth:~/practical_week9$ time ./copy_syscall source.txt test1.txt
File copied successfully using system calls.

real    0m0.001s
user    0m0.001s
sys     0m0.000s
navaneeth@Navaneeth:~/practical_week9$ time ./copy_standard source.txt test2.txt
File copied successfully using standard library functions.

real    0m0.002s
user    0m0.002s
sys     0m0.001s
navaneeth@Navaneeth:~/practical_week9$ |

```

Observation

- open() opens the file.
- read() reads data from the source file.
- write() writes data to the destination file.
- close() closes the files.
- The standard library version uses fopen(), fread(), fwrite(), and fclose().
- Both programs successfully copy the file.
- cmp verifies that the copied files are identical.
- time can be used to compare their execution time.

Result

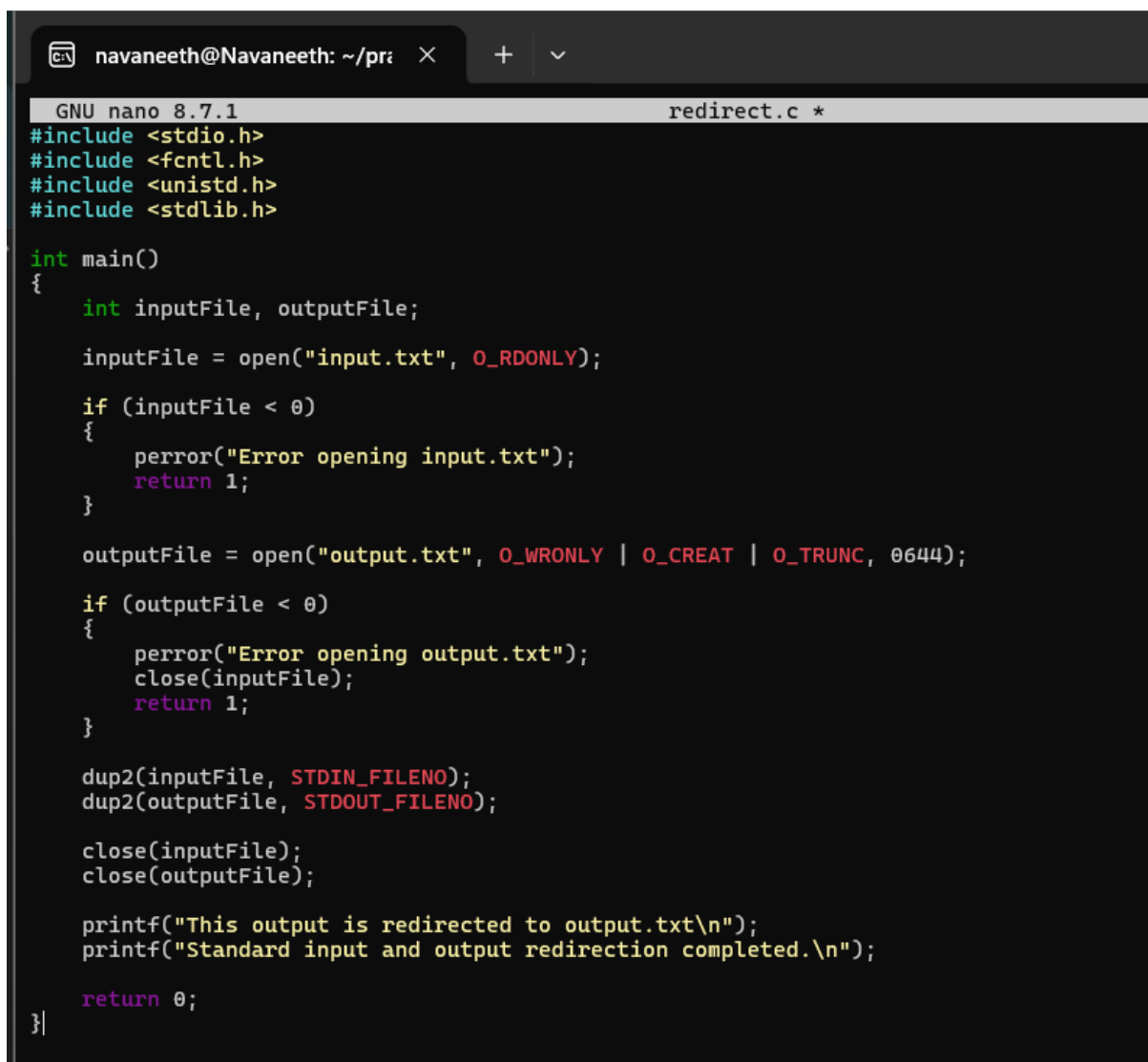
A file-copy utility was successfully implemented using low-level Linux system calls and compared with a standard C library implementation.

When you finish this, say **next** and we'll do **Practical Week 9 – Question 2: Standard Input/Output Redirection using dup2()**.

Aim

To understand file descriptors and implement input and output redirection using the Linux dup2() system call.

Program:



```
GNU nano 8.7.1                                redirect.c *
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>

int main()
{
    int inputFile, outputFile;

    inputFile = open("input.txt", O_RDONLY);

    if (inputFile < 0)
    {
        perror("Error opening input.txt");
        return 1;
    }

    outputFile = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (outputFile < 0)
    {
        perror("Error opening output.txt");
        close(inputFile);
        return 1;
    }

    dup2(inputFile, STDIN_FILENO);
    dup2(outputFile, STDOUT_FILENO);

    close(inputFile);
    close(outputFile);

    printf("This output is redirected to output.txt\n");
    printf("Standard input and output redirection completed.\n");

    return 0;
}
```

```

navaneeth@Navaneeth:~/practical_week9$ ls
copied_standard.txt  copy_standard  copy_syscall  source.txt  test2.txt
copied_syscall.txt  copy_standard.c  copy_syscall.c  test1.txt
navaneeth@Navaneeth:~/practical_week9$ nano redirect.c
navaneeth@Navaneeth:~/practical_week9$
navaneeth@Navaneeth:~/practical_week9$ echo "Hello Linux" > input.txt
navaneeth@Navaneeth:~/practical_week9$ echo "This is standard input redirection." >> input.txt
navaneeth@Navaneeth:~/practical_week9$ cat input.txt
Hello Linux
This is standard input redirection.
navaneeth@Navaneeth:~/practical_week9$ gcc redirect.c -o redirect
navaneeth@Navaneeth:~/practical_week9$ ls
copied_standard.txt  copy_standard  copy_syscall  input.txt  redirect.c  test1.txt
copied_syscall.txt  copy_standard.c  copy_syscall.c  redirect  source.txt  test2.txt
navaneeth@Navaneeth:~/practical_week9$ ./redirect
navaneeth@Navaneeth:~/practical_week9$ cat output.txt
This output is redirected to output.txt
Standard input and output redirection completed.
navaneeth@Navaneeth:~/practical_week9$ dup2(inputFile, STDIN_FILENO);
dup2(outputFile, STDOUT_FILENO);
-bash: syntax error near unexpected token `inputFile,'
-bash: syntax error near unexpected token `outputFile,'
navaneeth@Navaneeth:~/practical_week9$ nano redirect.c
navaneeth@Navaneeth:~/practical_week9$ gcc redirect.c -o redirect
navaneeth@Navaneeth:~/practical_week9$ ./redirect
navaneeth@Navaneeth:~/practical_week9$ cat output.txt
This output is redirected to output.txt
Standard input and output redirection completed.
navaneeth@Navaneeth:~/practical_week9$

```

Final Result

File copying using low-level system calls, comparison with standard library functions, and standard I/O redirection using `dup2()` were successfully implemented and verified.